Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
Wydział Fizyki i Informatyki Stosowanej

Informatyka Stosowana, studia magisterskie
Systemy/Programowanie Równoległe i Rozproszone

Repozytorium projektu na GitHub

Drzewa wszystkich najkrótszych ścieżek

Algorytm Dijkstry — implementacja MPI

wg Grama, Gupta – „Introduction to Parallel Computing", par. 10.3

Projekt nr 1 – aplikacja równoległa MPI

Autorzy: *Jakub Szydełko*
Jakub Urbański

Kraków, 2025/2026

Spis treści

1 Wprowadzenie **3**
1.1 Cel projektu . . . 3
1.2 Sformułowanie problemu . . . 3

2 Algorytm Dijkstry **4**
2.1 Wersja sekwencyjna . . . 4
2.2 Złożoność wersji sekwencyjnej . . . 4

3 Zrównoleglenie wg Gramy/Gupty (par. 10.3) **5**
3.1 Dwa podejścia do problemu wszystkich par . . . 5
3.2 Wybór formuły . . . 5
3.3 Dekompozycja danych . . . 5
3.4 Schemat komunikacji MPI . . . 6
3.5 Złożoność i analiza skalowania . . . 6

4 Architektura projektu **7**
4.1 Struktura repozytorium . . . 7
4.2 Format danych wejściowych . . . 7
4.3 Format danych wyjściowych . . . 8

5 Schemat blokowy programu **9**
5.1 Schemat głównej pętli programu . . . 9
5.2 Schemat wewnętrznego przebiegu Dijkstry . . . 10

6 Opis implementacji **11**
6.1 Struktura kodu źródłowego . . . 11
6.2 Główny przepływ programu . . . 11
6.3 Format pliku wynikowego JSON . . . 12
6.4 Kluczowe fragmenty implementacji . . . 12
6.4.1 Inicjalizacja i rozesłanie danych . . . 12
6.4.2 Selekcja globalnego minimum . . . 13
6.4.3 Akumulacja wyników wszystkich źródeł . . . 13

7 Budowa i obsługa programu **14**
7.1 Wymagania środowiskowe . . . 14
7.2 Konfiguracja środowiska . . . 14
7.3 Reguły Makefile . . . 14
7.4 Uruchamianie – krok po kroku . . . 15
7.4.1 Krok 1 – połączenie z węzłem klastra . . . 15
7.4.2 Krok 2 – konfiguracja środowiska . . . 15
7.4.3 Krok 3 – kompilacja . . . 15
7.4.4 Krok 4 – uruchomienie . . . 15

8 Przykład działania **16**

9 Aplikacja webowa do wizualizacji wyników **17**
9.1 Funkcjonalność . . . 17
9.2 Architektura . . . 18
9.3 Uruchamianie . . . 18
9.3.1 Krok 1 – środowisko Pythona . . . 18
9.3.2 Krok 2 – konfiguracja MPI i uruchomienie serwera . . . 18

9.3.3 Krok ewentualny – przekierowanie portu z maszyny lokalnej 19
9.4 Status i ograniczenia 19

1. Wprowadzenie

1.1. Cel projektu

Przedmiotem projektu jest równoległa implementacja algorytmu Dijkstry wyznaczającego **drzewa wszystkich najkrótszych ścieżek** w grafie ważonym, tj. dla każdego wierzchołka źródłowego $s \in V$ wyznaczenie:

- długości najkrótszych ścieżek $l(s, v)$ do wszystkich pozostałych wierzchołków $v \in V$,
- drzewa najkrótszych ścieżek o korzeniu s, reprezentowanego tablicą poprzedników `parent[v]`.

Zrównoleglenie zostało przeprowadzone z wykorzystaniem standardu **MPI** (implementacja MPICH) zgodnie z formułą *source-parallel* opisaną w rozdziale 10.3 podręcznika Gramy, Karypisa, Kumara i Gupty [1].

1.2. Sformułowanie problemu

Niech $G = (V, E, w)$ będzie skierowanym (lub nieskierowanym) grafem ważonym, gdzie $|V| = n$, a $w\colon E \to \mathbb{Z}_{>0}$ jest funkcją wagi krawędzi. Wagi przyjmujemy nieujemne, co jest warunkiem koniecznym poprawności algorytmu Dijkstry.

Wejście: macierz sąsiedztwa $W \in \mathbb{Z}^{n \times n}$, przy czym $W_{ij} > 0$ oznacza krawędź (i, j) o wadze W_{ij}, a $W_{ij} = -1$ – brak krawędzi.

Wyjście: dla każdego $s \in \{0, 1, \ldots, n-1\}$:

$$l(s, \cdot) \in (\mathbb{Z}_{\geq 0} \cup \{\infty\})^n, \qquad \pi(s, \cdot) \in (V \cup \{-1\})^n,$$

gdzie $l(s, v)$ to długość najkrótszej ścieżki $s \to v$, a $\pi(s, v)$ to bezpośredni poprzednik v na tej ścieżce (lub -1, jeśli ścieżka nie istnieje, lub $v = s$).

2. Algorytm Dijkstry

2.1. Wersja sekwencyjna

Klasyczny algorytm Dijkstry rozszerza zbiór V_T wierzchołków o znanej najkrótszej odległości od s jeden po drugim, w kolejności rosnących odległości. W każdym kroku wybierany jest wierzchołek $u \in V \setminus V_T$ o minimalnej tymczasowej etykiecie $l(u)$, a następnie etykiety jego sąsiadów są *relaksowane*.

```
DIJKSTRA(G, w, s):
  for v in V:
      l[v]     <- INF
      parent[v]<- -1
      visited[v] <- false
  l[s] <- 0
  while exists v: not visited[v] and l[v] < INF:
      u <- argmin { l[v] : not visited[v] }     # selekcja
      visited[u] <- true
      for each neighbour v of u:                # relaksacja
          if not visited[v] and l[u] + w(u,v) < l[v]:
              l[v]     <- l[u] + w(u,v)
              parent[v] <- u
```

Listing 1: Pseudokod algorytmu Dijkstry.

2.2. Złożoność wersji sekwencyjnej

Naiwna implementacja (bez kolejki priorytetowej, z liniowym wyszukiwaniem minimum) ma złożoność czasową $\mathcal{O}(n^2)$ dla pojedynczego źródła i $\mathcal{O}(n^3)$ dla problemu wszystkich par. Wybór tej wersji jest typowy dla zrównoleglenia macierzowego i przedstawiony w Gramie/Gupcie [1, par. 10.3]: jest prosty, nie wymaga rozproszonej struktury kopcowej, a jego dwie zagnieżdżone fazy (selekcja minimum + relaksacja) bardzo dobrze poddają się dekompozycji danych.

3. Zrównoleglenie wg Gramy/Gupty (par. 10.3)

3.1. Dwa podejścia do problemu wszystkich par

W rozdziale 10.3 [1] dla problemu wyznaczania najkrótszych ścieżek dla wszystkich źródeł autorzy wyróżniają dwie formuły zrównoleglenia algorytmu Dijkstry:

Source-Partitioned – partycjonowanie źródeł. Zbiór źródeł V jest dzielony pomiędzy p procesów (każdy otrzymuje n/p źródeł), a każdy proces wykonuje *niezależnie* pełny sekwencyjny algorytm Dijkstry dla swoich źródeł. Po inicjalnym rozesłaniu macierzy wagi nie ma żadnej dalszej komunikacji. Złożoność: $\mathcal{O}(n^3/p)$, skalowanie ograniczone do $p \leq n$.

Source-Parallel – zrównoleglenie pojedynczego przebiegu. Każdy pojedynczy przebieg algorytmu Dijkstry jest zrównoleglony przez podział zbioru wierzchołków V pomiędzy procesy. Główna pętla po źródłach $s = 0, \ldots, n-1$ wykonywana jest sekwencyjnie, natomiast wewnątrz każdego przebiegu wszystkie procesy współpracują. Złożoność: $\mathcal{O}(n^3/p + n^2 \log p)$, skalowanie do $p \leq n^2/\log n$.

3.2. Wybór formuły

Zaimplementowana została **formuła Source-Parallel**. Powody wyboru:

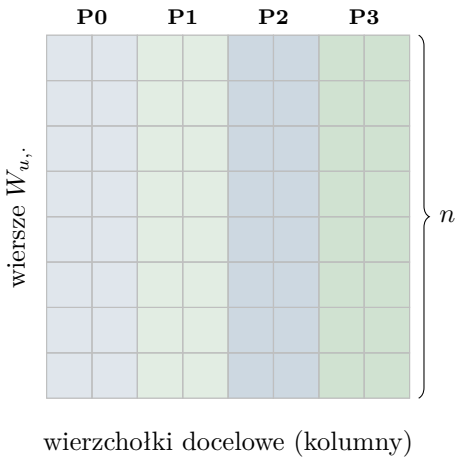
- zapewnia lepsze skalowanie dla typowych konfiguracji klastra (gdzie p może być porównywalne z n),
- pozwala wykorzystać liczbę procesów $p > n$ tam, gdzie formuła źródłowa traci sens,
- dobrze ilustruje wykorzystanie kolektywnej redukcji `MPI_Allreduce` z operatorem `MPI_MINLOC`, co jest jednym z kanonicznych wzorców komunikacyjnych MPI.

3.3. Dekompozycja danych

Niech p będzie liczbą procesów. Każdy proces o randze $r \in \{0, \ldots, p-1\}$ otrzymuje blok wierzchołków:

$$V_r = \left[\, \frac{r \cdot n}{p},\ \frac{(r+1) \cdot n}{p} \,\right).$$

Każdy proces przechowuje wprawdzie pełne tablice $l[\cdot]$, `parent`$[\cdot]$, `visited`$[\cdot]$ rozmiaru n (co upraszcza implementację), ale *aktualizuje* wyłącznie wpisy odpowiadające swojemu blokowi V_r. Dzięki temu każda iteracja zewnętrznej pętli (po krokach algorytmu) wymaga tylko jednej redukcji globalnej.



Rysunek 1: Podział wierzchołków pomiędzy $p = 4$ procesy (formuła source-parallel). Każdy proces P_r utrzymuje wpisy $l[v]$, `parent`$[v]$ dla $v \in V_r$ i podczas relaksacji korzysta z wiersza W_{u,V_r} macierzy wagi.

3.4. Schemat komunikacji MPI

Komunikacja między procesami sprowadza się do trzech wzorców:

1. **Inicjalizacja danych** – proces o randze 0 wczytuje macierz W z pliku, po czym rozsyła rozmiar n oraz całą macierz do pozostałych procesów:

```
MPI_Bcast(&n,    1,     MPI_INT, 0, MPI_COMM_WORLD);
MPI_Bcast(W.data(), n*n, MPI_INT, 0, MPI_COMM_WORLD);
```

2. **Globalne minimum z lokalizacją** – w każdym kroku wewnętrznej pętli wszystkie procesy wymieniają lokalne pary `(dist,vertex)` i wybierają parę o minimalnej odległości (przy remisach – o mniejszym indeksie) używając operatora `MPI_MINLOC`:

```
MPI_Allreduce(&localMin, &globalMin, 1,
              MPI_2INT, MPI_MINLOC, MPI_COMM_WORLD);
```

3. **Zebranie wyników po źródle** – po zakończeniu przebiegu Dijkstry dla danego s proces 0 otrzymuje pełne wektory l i `parent` z każdego procesu (każdy wkłada swój blok rozmiaru $|V_r|$):

```
MPI_Gatherv(l.data() + start, end - start, MPI_INT,
            finalL.data(), counts.data(), displs.data(),
            MPI_INT, 0, MPI_COMM_WORLD);
```

3.5. Złożoność i analiza skalowania

Dla każdego z n źródeł wykonujemy:

- $\mathcal{O}(n/p)$ pracy w kroku selekcji lokalnego minimum, $\mathcal{O}(n)$ kroków – razem $\mathcal{O}\big(n^2/p\big)$;
- $\mathcal{O}(n/p)$ pracy w relaksacji, $\mathcal{O}(n)$ kroków – razem $\mathcal{O}\big(n^2/p\big)$;
- $\mathcal{O}(n)$ wywołań `MPI_Allreduce`, każde o koszcie $\mathcal{O}(\log p)$ – razem $\mathcal{O}(n \log p)$.

Po pomnożeniu przez liczbę źródeł n otrzymujemy całkowity koszt:

$$T(n,p) \;=\; \mathcal{O}\Big(\tfrac{n^3}{p} + n^2 \log p\Big)\,.$$

Składnik komunikacyjny $n^2 \log p$ ogranicza efektywne skalowanie do $p = \mathcal{O}(n^2/\log n)$, co jest zgodne z analizą przedstawioną w [1, par. 10.3].

4. Architektura projektu

4.1. Struktura repozytorium

Projekt zorganizowany jest w trzy podkatalogi tematyczne. Trzon projektu MPI – zgodnie ze specyfikacją – znajduje się w katalogu `1_MPI/` i stanowi samodzielną, kompletną aplikację. Pozostałe katalogi zawierają narzędzia pomocnicze: skrypty wsadowe oraz graficzny interfejs do wizualizacji wyników.

```
SRiR-2026-Dijkstra/
|-- 1_MPI/                      # aplikacja MPI -- trzon projektu
|   |-- src/
|   |   `-- DijkstraMPI.cpp     # kod zrodlowy aplikacji rownoleglej
|   |-- data/
|   |   |-- matrix_4.txt        # przykladowa macierz wejsciowa
|   |   |-- matrix_4.json       # wynik (generowany podczas uruchomienia)
|   |   `-- ...                 # inne wygenerowane macierze testowe
|   |-- build/                  # (tworzony przez `make`)
|   |   `-- DijkstraMPI         # plik wykonywalny
|   `-- Makefile                # reguly kompilacji / uruchamiania / czyszczenia
|-- scripts/                    # skrypty pomocnicze (Python)
`-- webapp/                     # interfejs graficzny do wizualizacji wynikow
```

Katalog `scripts/` zawiera narzędzia pomocnicze niezbędne do przygotowania danych testowych oraz alternatywnych ścieżek uruchamiania – przede wszystkim generator losowych macierzy spójnych grafów ważonych.

Katalog `webapp/` zawiera prostą aplikację Flask udostępniającą interaktywną wizualizację drzew najkrótszych ścieżek wyznaczonych przez program MPI; jej architektura i obsługa opisane są krótko w osobnym rozdziale.

W dalszej części dokumentacji koncentrujemy się na zawartości katalogu `1_MPI/` – to właśnie ten podkatalog stanowi deliverable projektu zgodnie ze specyfikacją.

4.2. Format danych wejściowych

Plik wejściowy zawiera pełną macierz sąsiedztwa W rozmiaru $n \times n$ w formacie tekstowym:

- każdy wiersz pliku odpowiada jednemu wierszowi macierzy,
- wartości w wierszu są oddzielone znakami tabulacji (`\t`),
- każda wartość to -1 albo liczba całkowita dodatnia,
- $W_{ij} = -1$ oznacza brak krawędzi (w szczególności $W_{ii} = -1$, brak samopętli),
- $W_{ij} > 0$ oznacza krawędź $i \to j$ o wadze W_{ij}.

Przykład – graf 4-wierzchołkowy (`data/matrix_4.txt`):

```
-1      -1      10      16
-1      -1       8      -1
10       8      -1      -1
16      -1      -1      -1
```

Generowanie losowych macierzy testowych odbywa się skryptem `scripts/random_matrix_generator.py`, który gwarantuje spójność wynikowego grafu (najpierw losowe drzewo rozpinające, następnie dodatkowe krawędzie do osiągnięcia zadanej gęstości).

4.3. Format danych wyjściowych

Program produkuje wyniki w dwóch postaciach:

Standardowe wyjście (stdout) – wypisywane wyłącznie przez proces o randze 0. Dla każdego źródła $s \in \{0, \ldots, n-1\}$ i każdego wierzchołka docelowego v pokazana jest:

- długość najkrótszej ścieżki (`INF` – jeśli wierzchołek nieosiągalny),
- ścieżka jako sekwencja wierzchołków $s \to \cdots \to v$ odtworzona z tablicy poprzedników.

```
Shortest paths from source 0:
0 -> 0: 0  | path: 0
0 -> 1: 18 | path: 0 2 1
0 -> 2: 10 | path: 0 2
0 -> 3: 16 | path: 0 3
```

Plik JSON – generowany automatycznie obok pliku z macierzą wejściową (np. `data/matrix_4.txt` → `data/matrix_4.json`), zawiera wyniki dla *wszystkich* źródeł oraz statystyki czasu wykonania:

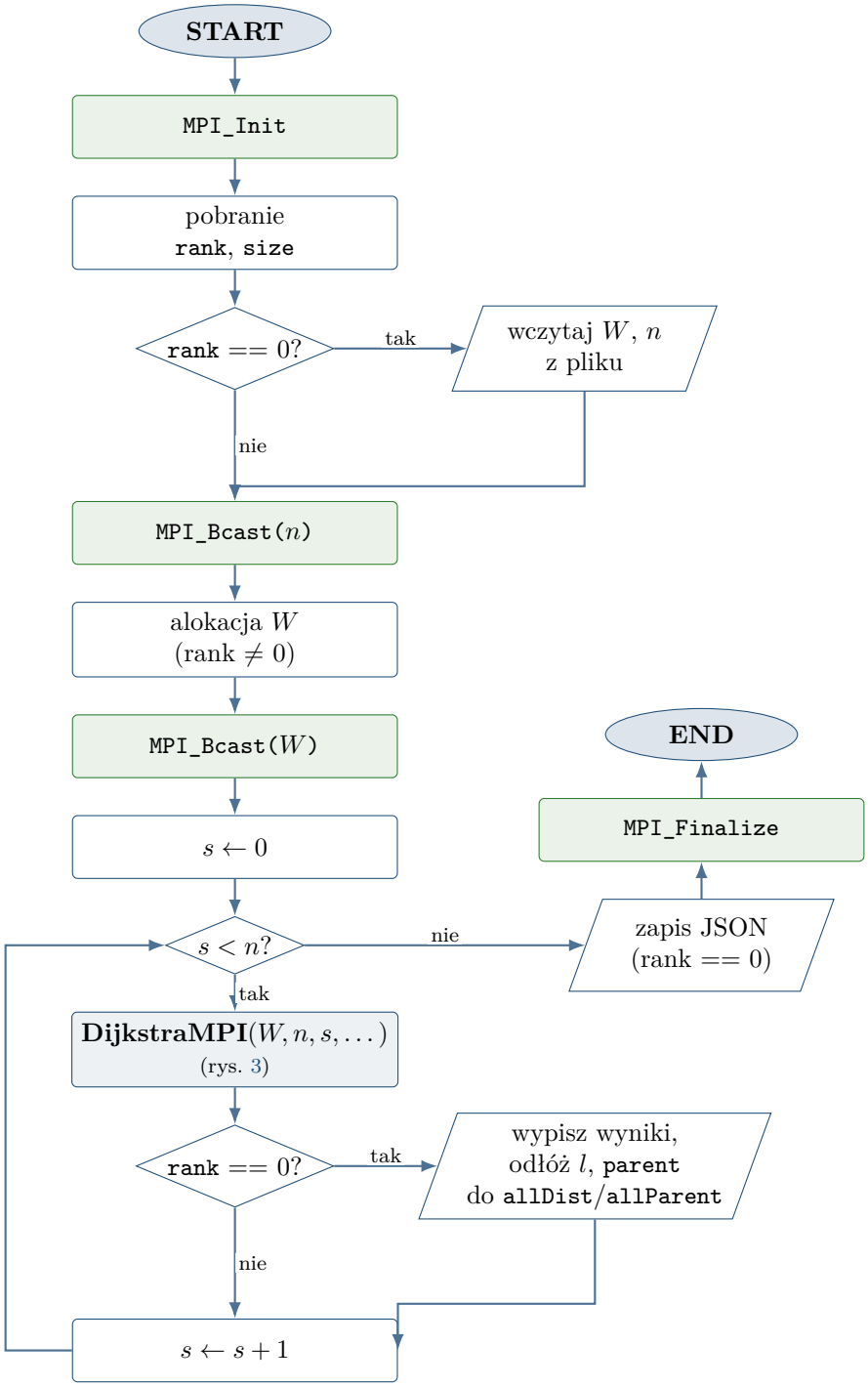
```
{
  "n": 4,
  "num_processes": 16,
  "io_seconds":      0.0145,
  "compute_seconds": 0.0023,
  "elapsed_seconds": 0.0168,
  "dist":   [ [...], [...], ... ],   // dist[s][v]   = l(s, v)
  "parent": [ [...], [...], ... ]    // parent[s][v] = pi(s, v)
}
```

Plik JSON jest wykorzystywany przez aplikację webową do prezentacji graficznej drzew najkrótszych ścieżek; jego format opisany jest szczegółowo w sekcji 6.3.

5. Schemat blokowy programu

5.1. Schemat głównej pętli programu

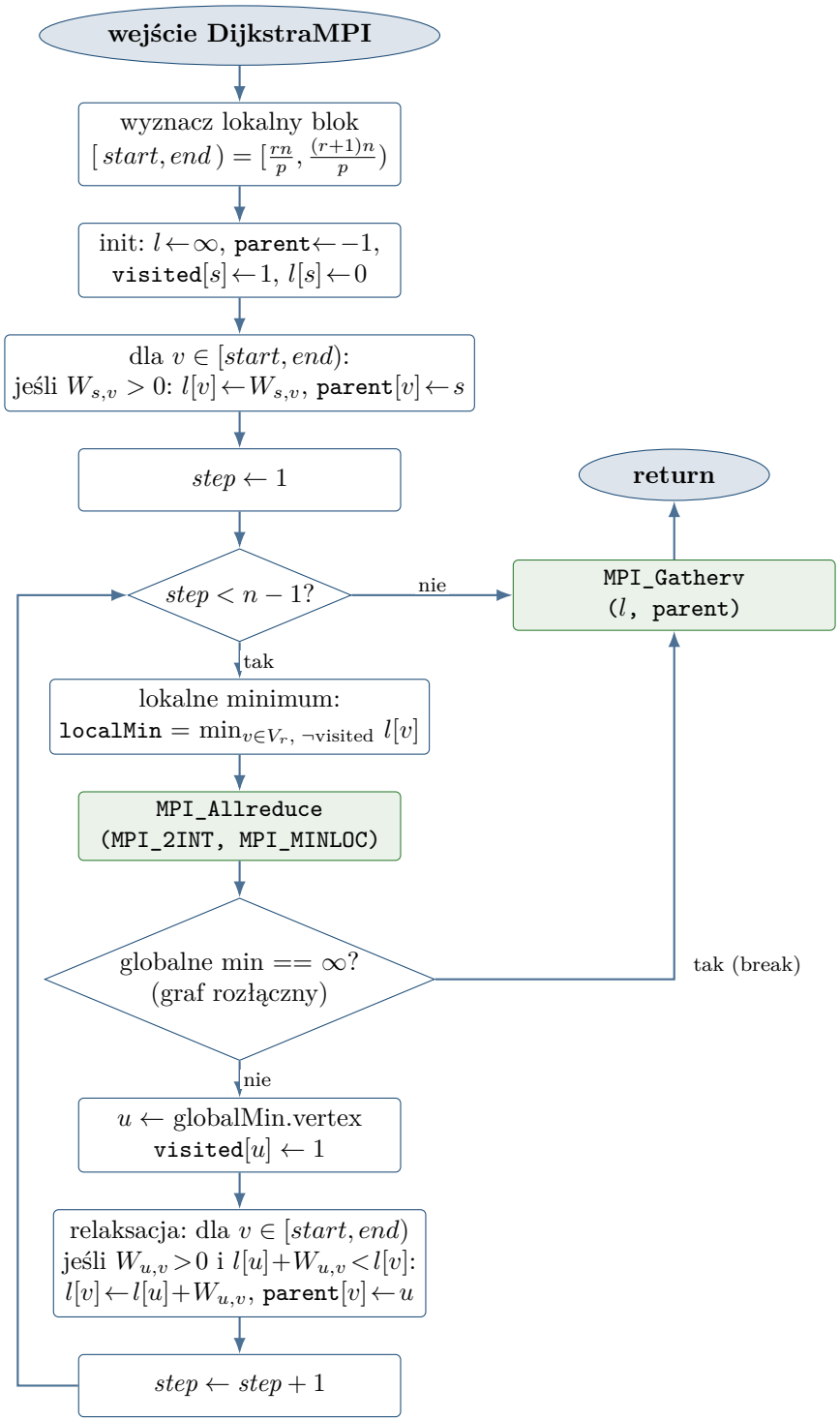
Schemat na rys. 2 przedstawia ogólny przepływ sterowania w programie – od inicjalizacji środowiska MPI, przez rozesłanie danych wejściowych, główną pętlę po źródłach, akumulację wyników, zapis pliku JSON, aż do finalizacji. Bloki w kolorze zielonym oznaczają wywołania funkcji MPI; blok jasnoniebieski reprezentuje wywołanie funkcji `DijkstraMPI` szczegółowo opisanej na rys. 3.



Rysunek 2: Schemat blokowy głównej pętli programu `DijkstraMPI`. Bloki w kolorze zielonym oznaczają wywołania funkcji MPI. Po zakończeniu pętli po źródłach proces 0 zapisuje zbiorczy plik JSON zawierający drzewa najkrótszych ścieżek dla wszystkich źródeł.

5.2. Schemat wewnętrznego przebiegu Dijkstry

Rys. 3 pokazuje w detalu funkcję `DijkstraMPI(W, n, s, rank, size)` – pojedynczy przebieg algorytmu Dijkstry zrównoleglony pomiędzy p procesów zgodnie z formułą source-parallel z rozdziału 10.3 podręcznika Gramy i Gupty [1]. Bloki w kolorze zielonym oznaczają synchroniczne wywołania kolektywne MPI; pętla wewnętrzna (po krokach algorytmu) realizuje sekwencję „lokalne minimum → globalna redukcja → relaksacja”.



Rysunek 3: Schemat blokowy funkcji `DijkstraMPI` – pojedynczy przebieg algorytmu Dijkstry zrównoleglony przez podział wierzchołków zgodnie z formułą source-parallel. Bloki zielone oznaczają synchroniczne wywołania kolektywne MPI.

6. Opis implementacji

6.1. Struktura kodu źródłowego

Cały kod znajduje się w pliku `src/DijkstraMPI.cpp`. Wyróżnione zostały następujące funkcje:

Funkcja	Rola
`ImportGraphMatrix`	Wczytuje macierz W z pliku tekstowego; wywoływana wyłącznie przez proces 0.
`DijkstraMPI`	Wykonuje pojedynczy przebieg równoległego algorytmu Dijkstry dla zadanego źródła s; zwraca pełne wektory l i `parent` (zebrane na procesie 0).
`PrintPath`	Rekurencyjnie odtwarza ścieżkę z tablicy poprzedników i wypisuje ją na stdout.
`ParseArgs`	Prosty parser flag wiersza poleceń (ścieżka macierzy, opcjonalne `--output`).
`ResolveOutputPath`	Wyznacza docelową ścieżkę pliku JSON na podstawie argumentu `--output` oraz nazwy macierzy wejściowej.
`WriteJsonResult`	Zapisuje wyniki dla wszystkich źródeł w formacie JSON (bez zewnętrznych bibliotek).

Tabela 1: Funkcje programu `DijkstraMPI`.

6.2. Główny przepływ programu

Po inicjalizacji środowiska MPI program wykonuje cztery fazy:

1. **Wczytanie i rozesłanie macierzy** – proces 0 wczytuje plik z macierzą sąsiedztwa, po czym kolektywnym `MPI_Bcast` rozsyła rozmiar n i całą macierz W do pozostałych procesów. Czas tej fazy mierzony jest jako `io_seconds`.
2. **Pętla po źródłach** – dla każdego $s = 0, 1, \ldots, n-1$ wywoływana jest funkcja `DijkstraMPI(W, n, s, …)` wyznaczająca drzewo najkrótszych ścieżek o korzeniu s. Wyniki dla każdego źródła wypisywane są na stdout (przez proces 0) oraz odkładane do lokalnych wektorów `allDist` i `allParent`. Czas wszystkich n przebiegów mierzony jest jako `compute_seconds`.
3. **Zapis JSON** – proces 0 zapisuje zbiorczy plik wynikowy zawierający wszystkie drzewa najkrótszych ścieżek. Plik domyślnie powstaje obok pliku wejściowego (np. `data/matrix_4.txt` → `data/matrix_4.json`); ścieżkę można nadpisać przełącznikiem `--output`.
4. **Finalizacja** – `MPI_Finalize`.

Pomiary czasu wykonywane są przez `MPI_Wtime` wyłącznie na procesie 0 i zapisywane do JSON-a w trzech wariantach: `io_seconds` (wczytanie + dystrybucja danych), `compute_seconds` (cała pętla po źródłach), `elapsed_seconds` (suma).

6.3. Format pliku wynikowego JSON

Plik wynikowy ma strukturę zbiorczą – zawiera drzewa najkrótszych ścieżek dla *wszystkich* źródeł:

```
{
  "n": 4,
  "num_processes": 16,
  "io_seconds":      0.0145,
  "compute_seconds": 0.0023,
  "elapsed_seconds": 0.0168,
  "dist":   [ [...], [...], ... ],   // dist[s][v]   = l(s, v)
  "parent": [ [...], [...], ... ]    // parent[s][v] = pi(s, v)
}
```

Listing 2: Schemat pliku JSON.

W obu macierzach wpis -1 oznacza brak (nieosiągalność dla `dist`, brak poprzednika dla `parent`, w szczególności `parent[s][s] == -1`).

6.4. Kluczowe fragmenty implementacji

6.4.1. Inicjalizacja i rozesłanie danych

```
double tIoStart = MPI_Wtime();

std::vector<int> W;
int n = 0;

if (rank == 0)
    ImportGraphMatrix(args.matrixPath, W, n);

MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);

if (rank != 0)
    W.resize(n * n);

MPI_Bcast(W.data(), n * n, MPI_INT, 0, MPI_COMM_WORLD);

double ioTime = MPI_Wtime() - tIoStart;
```

Listing 3: Wczytanie macierzy i rozesłanie do wszystkich procesów.

Dwukrotny `Bcast` jest konieczny: rozmiar n rozsyłany jest najpierw, by procesy o randze $\neq 0$ mogły zaalokować bufor docelowy, a dopiero potem przesyłana jest cała macierz.

6.4.2. Selekcja globalnego minimum

```
struct {
    int dist;
    int vertex;
} localMin, globalMin;

localMin.dist   = INT_MAX;
localMin.vertex = -1;

for (int v = start; v < end; v++)
    if (!visited[v] && l[v] < localMin.dist) {
        localMin.dist   = l[v];
        localMin.vertex = v;
    }

MPI_Allreduce(&localMin, &globalMin, 1,
              MPI_2INT, MPI_MINLOC, MPI_COMM_WORLD);
```

Listing 4: Wyznaczenie wierzchołka o globalnie najmniejszej etykiecie.

Operator `MPI_MINLOC` na typie `MPI_2INT` traktuje pierwszą wartość jako klucz (minimalizowaną odległość), a drugą jako etykietę (numer wierzchołka). Przy remisach wybierany jest wierzchołek o mniejszym indeksie – co zapewnia deterministyczny wynik niezależnie od liczby procesów.

6.4.3. Akumulacja wyników wszystkich źródeł

```
std::vector<std::vector<int>> allDist;
std::vector<std::vector<int>> allParent;
if (rank == 0) {
    allDist.reserve(n);
    allParent.reserve(n);
}

for (int s = 0; s < n; s++) {
    auto result = DijkstraMPI(W, n, s, rank, size);

    if (rank == 0) {
        // ... wypisanie ścieek na stdout ...
        allDist.push_back(std::move(result.l));
        allParent.push_back(std::move(result.parent));
    }
}
```

Listing 5: Zapamiętanie wyniku każdego przebiegu na procesie 0.

Każdy z n przebiegów funkcji `DijkstraMPI` kończy się `MPI_Gatherv` – zebraniem lokalnych bloków wyników na procesie 0. Pełne wektory l i `parent` dla każdego źródła odkładane są następnie do zewnętrznych `allDist`/`allParent`, co pozwala zapisać kompletny zbiór drzew najkrótszych ścieżek do jednego pliku JSON po zakończeniu pętli.

7. Budowa i obsługa programu

7.1. Wymagania środowiskowe

Program przeznaczony jest do uruchamiania w środowisku pracowni komputerowej WFiIS AGH, na klastrze stacji roboczych `stud204-XX` (gdzie `XX` $\in \{00, 01, \ldots, 15\}$) udostępnianych przez serwer dostępowy `taurus.fis.agh.edu.pl`. Wymagane komponenty:

- kompilator C++ z obsługą standardu C++17,
- implementacja MPI w wersji 5.0 (`mpich-5.0.0` dostępna w pracowni przez skrypt `/opt/nfs/config/source_mpich500.sh`),
- GNU Make,
- dostęp do skryptu `station204_name_list.sh` (z katalogu `/opt/nfs/config/`) generującego listę węzłów obliczeniowych klastra.

7.2. Konfiguracja środowiska

Przed pierwszą kompilacją oraz przed każdą sesją pracy z programem należy załadować zmienne środowiskowe pakietu MPICH do aktualnej powłoki:

```
$ source /opt/nfs/config/source_mpich500.sh
$ which mpic++
/opt/nfs/mpich-5.0.0/bin/mpic++
```

Skrypt ustawia ścieżki PATH oraz zmienne wymagane do kompilacji i uruchamiania aplikacji MPI.

7.3. Reguły Makefile

Plik `Makefile` udostępnia trzy reguły wymagane przez specyfikację projektu:

Reguła	Wywołanie	Działanie
`all`	`make`	Kompilacja `src/DijkstraMPI.cpp` do `build/DijkstraMPI` przy użyciu `mpic++ -std=c++17 -O2`.
`run`	`make run`	Generuje listę 16 węzłów klastra (plik `nodes`), następnie uruchamia program z przykładowym plikiem danych `data/matrix_4.txt` na `NP` procesach.
`clean`	`make clean`	Usuwa katalog `build/` oraz wygenerowany plik `nodes` (przywraca podkatalog do stanu wyjściowego).
`reset`	`make reset`	Alias dla `clean`.

Tabela 2: Reguły pliku Makefile.

Liczbę procesów oraz plik wejściowy można nadpisać zmiennymi przy wywołaniu `make`:

```
$ make run NP=8
$ make run NP=16 DATA_FILE=data/matrix_100.txt
```

7.4. Uruchamianie – krok po kroku

Workflow dla typowej sesji pracy w pracowni obejmuje cztery etapy: zalogowanie się na węzeł obliczeniowy, konfigurację środowiska, kompilację i uruchomienie.

7.4.1. Krok 1 – połączenie z węzłem klastra

Połączenie zdalne odbywa się dwustopniowo: najpierw przez serwer dostępowy `taurus`, następnie na wybraną stację z listy `stud204-00` …`stud204-15`.

```
$ ssh -X user@taurus.fis.agh.edu.pl

user@taurus:~$ /opt/nfs/config/cluster204_test_nodes.sh
[0]  stud204-00: 172.20.204.0
[1]  stud204-01: 172.20.204.1
...
[15] stud204-15: 172.20.204.15

user@taurus:~$ ssh stud204-10
```

7.4.2. Krok 2 – konfiguracja środowiska

Po zalogowaniu na stację roboczą:

```
user@stud204-10:~$ source /opt/nfs/config/source_mpich500.sh
```

7.4.3. Krok 3 – kompilacja

```
user@stud204-10:~$ cd ś<żcieka>/SRiR-2026-Dijkstra/1_MPI
user@stud204-10:~/.../1_MPI$ make
mpic++ -std=c++17 -O2 -Wall -Wextra -o build/DijkstraMPI src/DijkstraMPI.cpp
```

7.4.4. Krok 4 – uruchomienie

```
user@stud204-10:~/.../1_MPI$ make run
/opt/nfs/config/station204_name_list.sh 1 16 > nodes
mpiexec -f nodes -n 16 ./build/DijkstraMPI data/matrix_4.txt

Shortest paths from source 0:
0 -> 0: 0  | path: 0
0 -> 1: 18 | path: 0 2 1
...

[JSON] wrote data/matrix_4.json (n=4, np=16, io=...s, compute=...s, total=...s)
```

Po zakończeniu działania programu wyniki dostępne są w dwóch formach: pełen log ścieżek na standardowym wyjściu oraz plik JSON `data/matrix_4.json` z kompletnym zbiorem drzew najkrótszych ścieżek. Sprzątanie środowiska:

```
user@stud204-10:~/.../1_MPI$ make clean
```

8. Przykład działania

Dla małego grafu testowego o 4 wierzchołkach:

$$W = \begin{pmatrix} -1 & -1 & 10 & 16 \\ -1 & -1 & 8 & -1 \\ 10 & 8 & -1 & -1 \\ 16 & -1 & -1 & -1 \end{pmatrix}$$

zwykle uruchomienie `mpiexec -n 4 ./build/DijkstraMPI data/example.txt` generuje:

```
Shortest paths from source 0:
0 -> 0: 0  | path: 0
0 -> 1: 4  | path: 0 1
0 -> 2: 6  | path: 0 1 2
0 -> 3: 1  | path: 0 3

Shortest paths from source 1:
1 -> 0: 4  | path: 1 0
1 -> 1: 0  | path: 1
1 -> 2: 2  | path: 1 2
1 -> 3: 5  | path: 1 0 3

Shortest paths from source 2:
2 -> 0: 6  | path: 2 1 0
2 -> 1: 2  | path: 2 1
2 -> 2: 0  | path: 2
2 -> 3: 7  | path: 2 1 0 3

Shortest paths from source 3:
3 -> 0: 1  | path: 3 0
3 -> 1: 5  | path: 3 0 1
3 -> 2: 7  | path: 3 0 1 2
3 -> 3: 0  | path: 3
```

Każda sekcja stanowi pełne drzewo najkrótszych ścieżek o zadanym korzeniu – co realizuje sformułowanie problemu „drzew wszystkich najkrótszych ścieżek”.

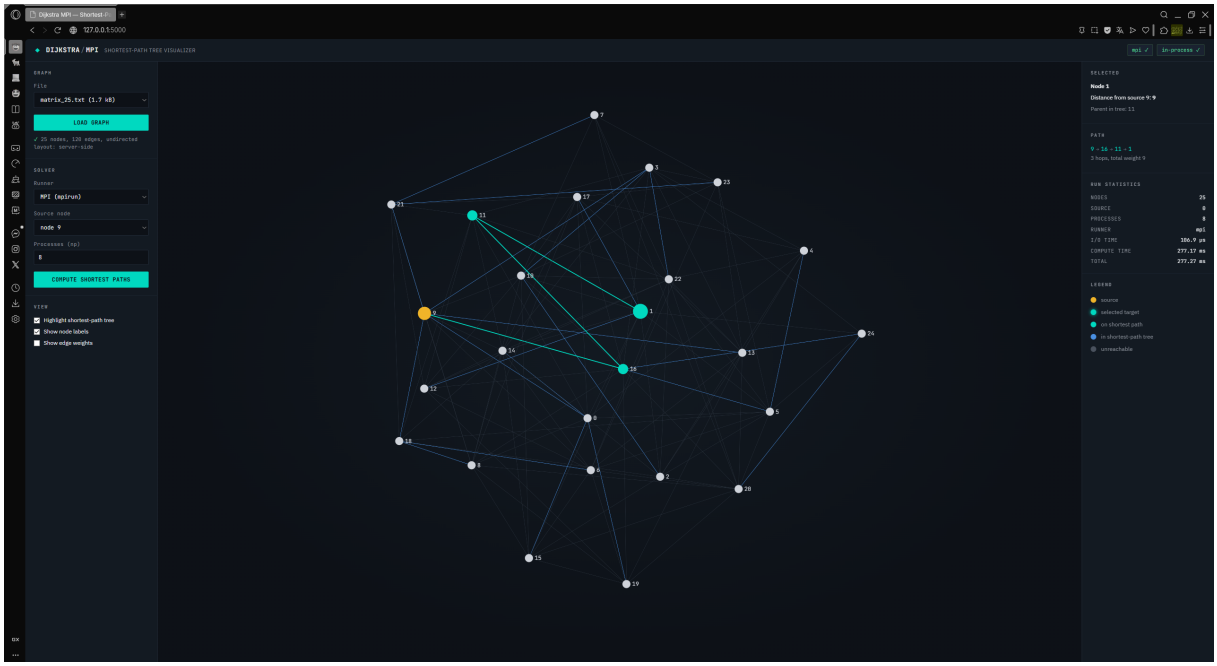
9. Aplikacja webowa do wizualizacji wyników

Na potrzeby graficznej prezentacji drzew najkrótszych ścieżek wyznaczonych przez program MPI powstała pomocnicza aplikacja webowa oparta na mikro-frameworku Flask, znajdująca się w katalogu `webapp/` repozytorium projektu. Aplikacja *nie jest* częścią deliverable wymaganego przez specyfikację projektu MPI – stanowi narzędzie pomocnicze, ułatwiające inspekcję wyników bez potrzeby parsowania surowego pliku JSON.

9.1. Funkcjonalność

Interfejs umożliwia:

- wybór i wczytanie macierzy wejściowej z katalogu `1_MPI/data/` (rozmieszczenie wierzchołków obliczane jest automatycznie przez algorytm Kamada-Kawai dla małych grafów, bądź ForceAtlas2 po stronie klienta dla większych),
- wywołanie programu `DijkstraMPI` dla wybranego pliku i zadanej liczby procesów (`NP`),
- interaktywne przełączanie wierzchołka źródłowego spośród n obliczonych drzew bez ponownego uruchamiania programu,
- podświetlanie drzewa najkrótszych ścieżek oraz wybór wierzchołka docelowego kliknięciem (z odtworzeniem ścieżki $s \to v$ na podstawie tablicy poprzedników),
- podgląd statystyk uruchomienia (czas I/O, czas obliczeń, czas całkowity, liczba procesów MPI).



Rysunek 4: Aplikacja webowa po wczytaniu macierzy testowej i wyznaczeniu drzew najkrótszych ścieżek. Złoty wierzchołek – aktualne źródło, niebieskie krawędzie – drzewo najkrótszych ścieżek o tym korzeniu, turkusowa ścieżka – najkrótsza droga do wybranego (kliknięciem) wierzchołka docelowego.

9.2. Architektura

Aplikacja składa się z dwóch warstw:

Backend (Flask) – plik `webapp/app.py`. Udostępnia REST API do listowania macierzy, parsowania wybranej macierzy i wyznaczania jej układu graficznego, oraz uruchamiania programu MPI w podprocesie. Wyniki czytane są bezpośrednio z pliku JSON wytwarzanego przez `DijkstraMPI`.

Frontend (SPA) – pliki `templates/index.html`, `static/css/style.css`, `static/js/app.js`. Pojedyncza strona rendrująca graf w bibliotekach *sigma.js* oraz *graphology*, komunikująca się z backendem przez `fetch`.

Backend uruchamia binarkę `DijkstraMPI` dokładnie tak samo jak typowe wywołanie `mpiexec` z linii poleceń – wymaga zatem tej samej konfiguracji środowiska MPI co uruchomienie „z ręki" (por. sek. 7.2).

9.3. Uruchamianie

Aplikacja przeznaczona jest do uruchamiania na węźle pracowni (`stud204-XX`), z przekierowaniem portu HTTP do maszyny lokalnej.

9.3.1. Krok 1 – środowisko Pythona

Aplikacja wymaga środowiska Python 3.10+ z zainstalowanymi zależnościami z `webapp/requirements.txt` (Flask, NetworkX, NumPy, SciPy). Środowisko można utworzyć przy użyciu Condy lub wbudowanego mechanizmu `venv` – obie opcje są równoważne, wystarczy wybrać jedną.

Wariant A – `venv` (standardowy moduł Pythona):

```
$ cd webapp/
$ python3 -m venv .venv
$ source .venv/bin/activate
$ pip install -r requirements.txt
```

Wariant B – Conda (zalecany, jeśli dostępny):

```
$ conda create -n srir python=3.11 -y
$ conda activate srir
$ cd webapp/
$ pip install -r requirements.txt
```

9.3.2. Krok 2 – konfiguracja MPI i uruchomienie serwera

W tym samym terminalu, w którym aktywne jest środowisko Pythona, należy załadować zmienne MPI (analogicznie jak w sek. 7.2):

```
$ source /opt/nfs/config/source_mpich500.sh
$ python3 app.py
 * Running on http://127.0.0.1:5000
```

9.3.3. Krok ewentualny – przekierowanie portu z maszyny lokalnej

Z komputera lokalnego (urządzenie użytkownika), w oddzielnym terminalu, otwiera się tunel SSH przez `taurus` z równoczesnym przekierowaniem portu:

```
$ ssh -J user@taurus.fis.agh.edu.pl -L 5000:localhost:5000 user@stud204-XX
```

Po zestawieniu tunelu aplikacja jest dostępna pod adresem `http://127.0.0.1:5000` w przeglądarce na maszynie lokalnej.

9.4. Status i ograniczenia

Aplikacja webowa stanowi narzędzie wewnętrzne projektu i nie była projektowana jako produkt produkcyjny. W szczególności: serwer Flask uruchamiany jest w trybie deweloperskim, brak jest mechanizmów uwierzytelnienia, a jednoczesna praca wielu użytkowników nie była testowana. W kontekście projektu MPI rola aplikacji ogranicza się do weryfikacji poprawności wyników i prezentacji drzew najkrótszych ścieżek – pełna i wymagana funkcjonalność programu dostępna jest również z poziomu wiersza poleceń (`make run`, sek. 7.4).

Literatura

[1] A. Grama, A. Gupta, G. Karypis, V. Kumar, *Introduction to Parallel Computing*, wyd. 2, Pearson / Addison-Wesley, 2003 (rozdz. 10.3 – All-Pairs Shortest Paths).

[2] *MPI: A Message-Passing Interface Standard*, ver. 3.1, Message Passing Interface Forum, 2015, https://www.mpi-forum.org/docs/.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Wprowadzenie do algorytmów*, wyd. 3, PWN, 2012 (rozdz. 24 – Najkrótsze ścieżki z jednego źródła).